

Implementación Cloud: Arquitectura Colaborativa en AWS

Mapeo de componentes conceptuales de ultra-baja latencia a servicios nativos de Amazon Web Services.

Destinatario: DevOps / Cloud Architects	Estado: Propuesta de Infraestructura	Cloud Provider: Amazon Web Services (AWS)	Clasificación: Confidencial Técnico
--	---	--	--

1. Introducción y Contexto del Despliegue

Este informe actúa como complemento al *Documento de Especificación Técnica de Sistemas Colaborativos en Tiempo Real*. Su objetivo es traducir la arquitectura agnóstica (Edge Termination, WebSockets de larga duración, Pub/Sub en memoria y almacenamiento evictivo) al ecosistema de servicios gestionados de **Amazon Web Services (AWS)**.

Diseñar sistemas distribuidos de tiempo real interactivo en la nube pública conlleva riesgos de "latencia artificial" si se seleccionan servicios orientados a microservicios tradicionales (como SQS, SNS o API Gateways sin estado). El enfoque aquí planteado maximiza el rendimiento y mantiene los costos operativos predecibles frente al escalado masivo de cursores y deltas de texto a 60 FPS.

2. Enrutamiento y Terminación de Red (El desafío del Borde)

Para cumplir el SLA de latencia de **< 200 ms p95** para clientes ubicados en regiones remotas o extremas (ej. Tierra del Fuego), es inaceptable enrutar el tráfico inicial de WebSocket a través de la Internet pública internacional.

- **Servicio Elegido:** **AWS Global Accelerator**.
- **Justificación:** Proporciona IPs estáticas *Anycast*. El cliente se conecta al PoP (Point of Presence) de AWS más cercano geográficamente, donde ingresa de inmediato al *backbone* privado de fibra óptica de AWS. Esto reduce drásticamente el *jitter*, minimiza los saltos BGP y acelera el Handshake TLS inicial del WebSocket.

3. Capa de Ingreso y Balanceo de Carga (WebSockets)

Existen dos enfoques nativos en AWS para ingerir tráfico WebSocket. La elección incorrecta puede resultar en fallos de conexión por límites rígidos o sobrecostos masivos.

Servicio AWS	Idoneidad	Análisis de Impacto (Pros & Cons)
Application Load Balancer (ALB)	Recomendado	Soporta conexiones TCP/WebSocket de muy larga duración de forma nativa. Latencia pura de <i>passthrough</i> . Costo basado en horas por LCU (Load Balancer Capacity Units) y ancho de banda, lo cual es predecible para tráfico de alta frecuencia y pequeño tamaño (cursores). Permite <i>Sticky Sessions</i> .
AWS API Gateway (WebSocket API)	Evitar (Trampa)	Factura por mensaje . Transmitir la posición de un cursor a 30-60 FPS generará millones de mensajes por usuario/hora, arruinando el presupuesto. Además, impone un <i>Hard Limit</i> de 2 horas tras las cuales corta la conexión forzosamente.

4. Capa de Cómputo (El "Collaboration Service")

Esta capa ejecuta el motor CRDT (ej. Yjs con Node.js, Rust o Go) que mantiene el estado del documento en memoria mientras los usuarios colaboran.

- **Servicio Elegido:** **Amazon ECS (Elastic Container Service) con AWS Fargate.**
- **Arquitectura de Contenedores:** Fargate exime al equipo de administrar máquinas virtuales EC2. Los contenedores se despliegan detrás del ALB. Como múltiples usuarios pueden estar en el mismo documento simultáneamente, el estado en memoria debe sincronizarse velozmente.
- **Sticky Sessions:** Aunque el ALB puede configurarse para intentar enrutar a los usuarios del mismo documento al mismo contenedor, en sistemas altamente distribuidos esto no siempre es garantizado. Por lo tanto, los contenedores **deben** depender de un Pub/Sub externo ultrarrápido para mantener consistencia.

Atención: Aislamiento del Bucle Principal

Asegúrese de perfilar las tareas de Fargate. Si se utiliza Node.js para el CRDT, el procesamiento de transformaciones matemáticas bloquea el Event Loop. Es vital no dimensionar los contenedores con menos de 2 vCPUs para permitir concurrencia entre red y cómputo.

5. El Motor de Tiempo Real en Memoria (Pub/Sub)

Para cumplir con los "Canales A y B" (Texto y Presencia) definidos en la arquitectura lógica, los contenedores de ECS requieren intercambiar los deltas y las coordenadas de cursores en tiempos sub-milisegundo.

- **Servicio Elegido:** **Amazon ElastiCache for Redis** (o alternativamente, el nuevo Amazon ElastiCache Serverless / Valkey).
- **Implementación:** Actúa exclusivamente como un broker de mensajes (usando Redis PUB/SUB). No almacena los datos de forma persistente. Cada documento activo tiene un canal de Redis asociado. Cuando el Contenedor A recibe un delta, lo publica en el canal del documento, y el Contenedor B, suscrito al mismo, recibe y despacha el delta a sus respectivos clientes vía WebSocket.

- **Anti-patrón:** No usar Amazon SQS, SNS, ni EventBridge para este propósito. Estos servicios orientados a eventos asíncronos introducen latencias inaceptables (decenas a cientos de ms) para el movimiento fluido de cursores.

6. Capa de Persistencia y Evicción (Cold Storage)

Conforme a la regla de "Persistencia por Evicción", los datos se bajan a bases estables solo cuando el documento queda inactivo.

- **Almacenamiento de Código/Snapshots:** **Amazon S3 (Simple Storage Service)**. Se almacenan las instantáneas consolidadas de los archivos binarios del CRDT y exportaciones de texto plano. S3 garantiza 99.999999999% de durabilidad.
- **Metadatos, ACLs y Chat (Canal C):** **Amazon Aurora Serverless v2 (PostgreSQL)**. Excelente para manejar picos de concurrencia cuando múltiples usuarios se unen e interactúan en el chat, escalando sus ACUs (Aurora Capacity Units) en milisegundos y reduciéndose cuando el sistema está en reposo.

7. Alternativa "Hacker" de Alta Escala: AWS IoT Core

Para escenarios donde la carga operativa de administrar contenedores con conexiones WebSocket abiertas (ECS/ALB) resulte abrumadora, existe una arquitectura serverless menos convencional pero extremadamente robusta: usar infraestructura diseñada originalmente para la *Internet de las Cosas*.



- **Cómo Funciona:** Los navegadores web se conectan directamente a **AWS IoT Core** utilizando el protocolo ligero MQTT sobre WebSockets (soportado nativamente por la SDK de AWS).
- **Ventaja Táctica:** IoT Core se convierte en el "Pub/Sub" administrado. Cuando un usuario mueve el cursor, publica un mensaje en el tópico MQTT (ej. doc/123/presence). IoT Core retransmite ese mensaje instantáneamente a los cientos de usuarios suscritos al tópico, sin que tengas que programar ni aprovisionar un solo servidor para manejar ese *Fan-Out*.
- **Trade-off:** Requiere configurar autenticación mediante Amazon Cognito o *Custom Authorizers* (Lambdas) para validar qué usuario tiene permisos para suscribirse a qué tópico (documento), pero reduce el mantenimiento de infraestructura al mínimo.